

Ensemble learning

Bagging & Boosting
weak learners

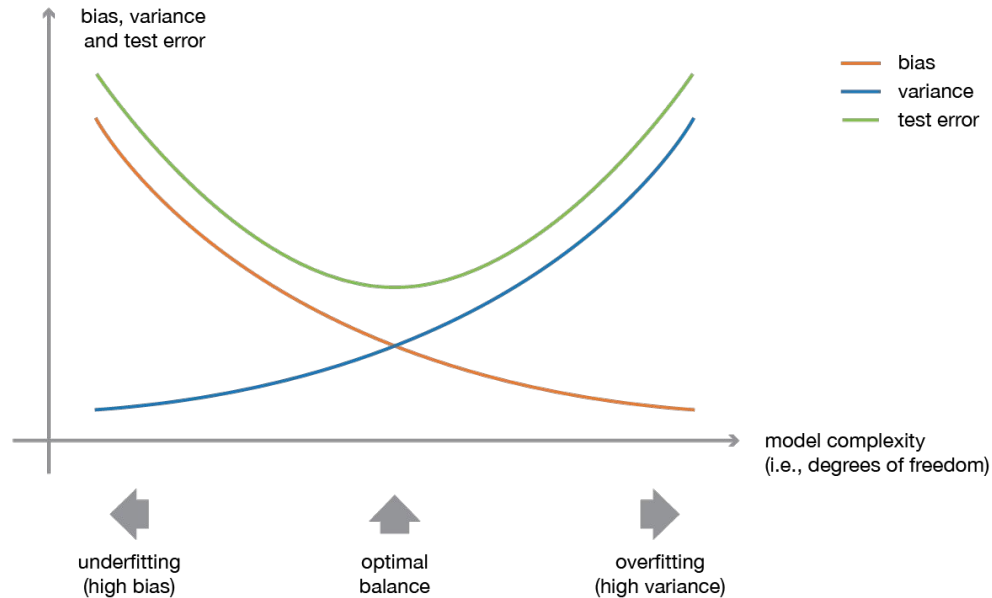
On the Menu

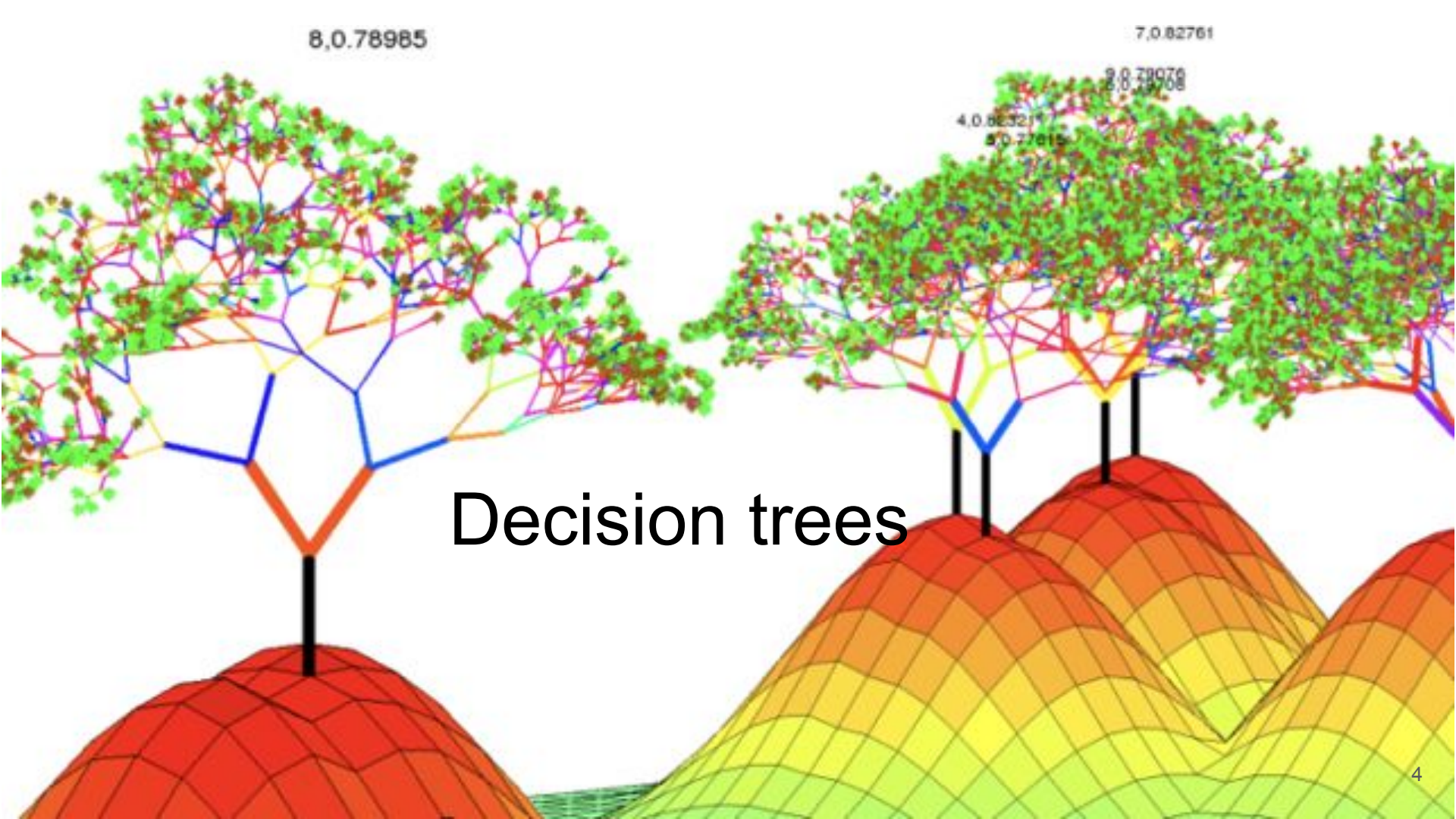
- bias variance tradeoff
- decision trees
- decision trees have high variance when unconstrained
- bagging can help
- random forests
- boosting
 - adaboost
- LR has high bias
- boosting can help GradientBoostingRegressor
- XGboost
-

Bias - Variance trade off

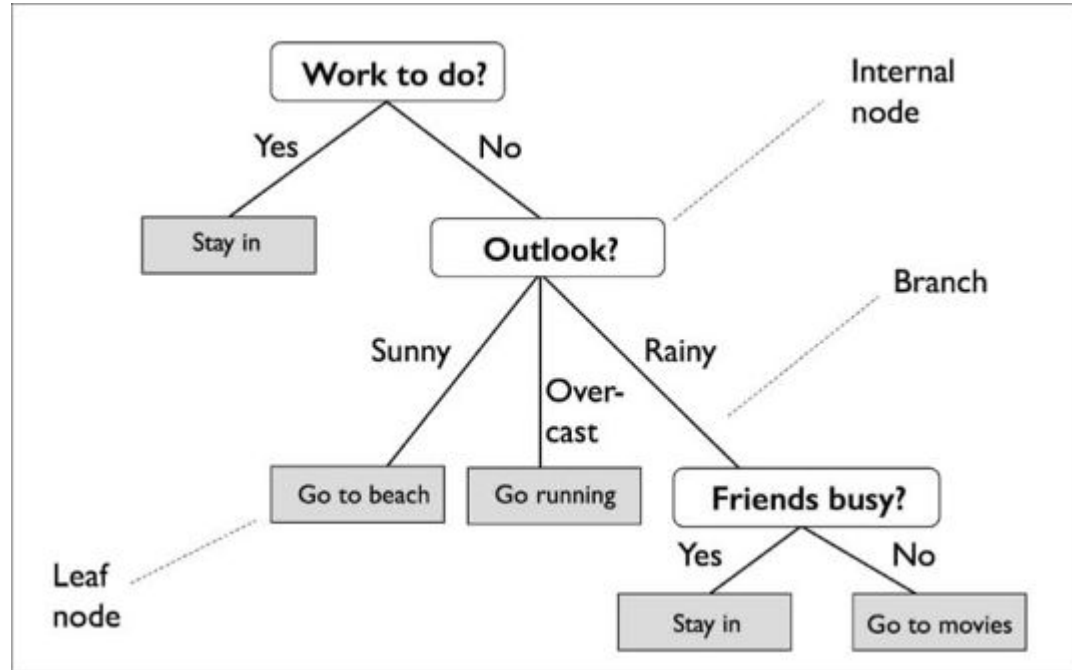
We want a model that performs well on unseen data

- low bias : good predictions
- low variance : not too sensitive to training data





Decision trees



Decision trees

Non parametric:

The structure (like the number of nodes or depth in a Decision Tree) is determined from the data itself.

Decision Trees (DTs) are a non-parametric supervised learning method used for classification and regression.

The goal is to create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features.

- *parametric* would be linear regression, linear models, or neural networks with fixed structure
- *non parametric* means that the model adapts to the data.

Parametric / non parametric models

Parametric models have a fixed number of parameters that are learned from the training data, and the model structure is defined before seeing any data.

Linear models like linear regression, logistic regression, and linear SVMs have fixed parameters (typically weights and bias terms) that define a linear decision boundary or function.

Non-parametric models, on the other hand, don't make strong assumptions about the form of the mapping function and can increase their complexity (often by using more parameters) as more training data becomes available.

Examples include k-nearest neighbors, decision trees, and kernel methods.

Greedy models

- Algorithms that make locally optimal decisions at each step
- Don't consider the global impact of these decisions
- Focus on immediate gain rather than long-term optimization

Common Examples

- **Decision Trees** (ID3, C4.5, CART): Select best split at each node
- **Boosting Algorithms** (AdaBoost, Gradient Boosting): Sequentially add models to minimize current error
- **Feature Selection**: Forward/backward selection methods that iteratively add/remove features

Is SGD greedy ?

SGD represents a form of "controlled greediness" that balances immediate gains with the possibility of finding better solutions

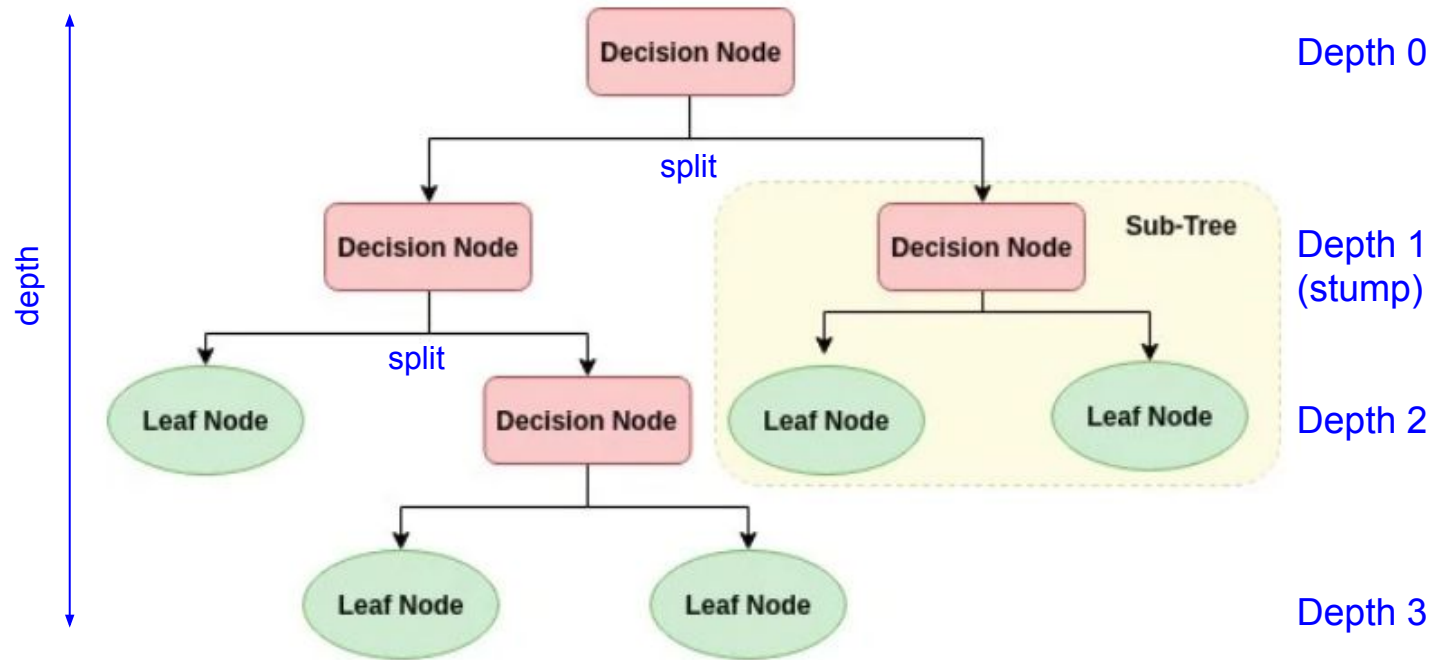
Greedy Characteristics

- Makes locally optimal decisions at each step
- Follows the negative gradient direction
- Aims to immediately reduce the loss function
- No explicit planning for future steps

Beyond Simple Greediness

- **Stochasticity:** Random sampling helps escape local minima
- **Learning Rate:** Controls "how greedy" the algorithm behaves
- **Momentum:** Considers past gradients, reducing pure greediness

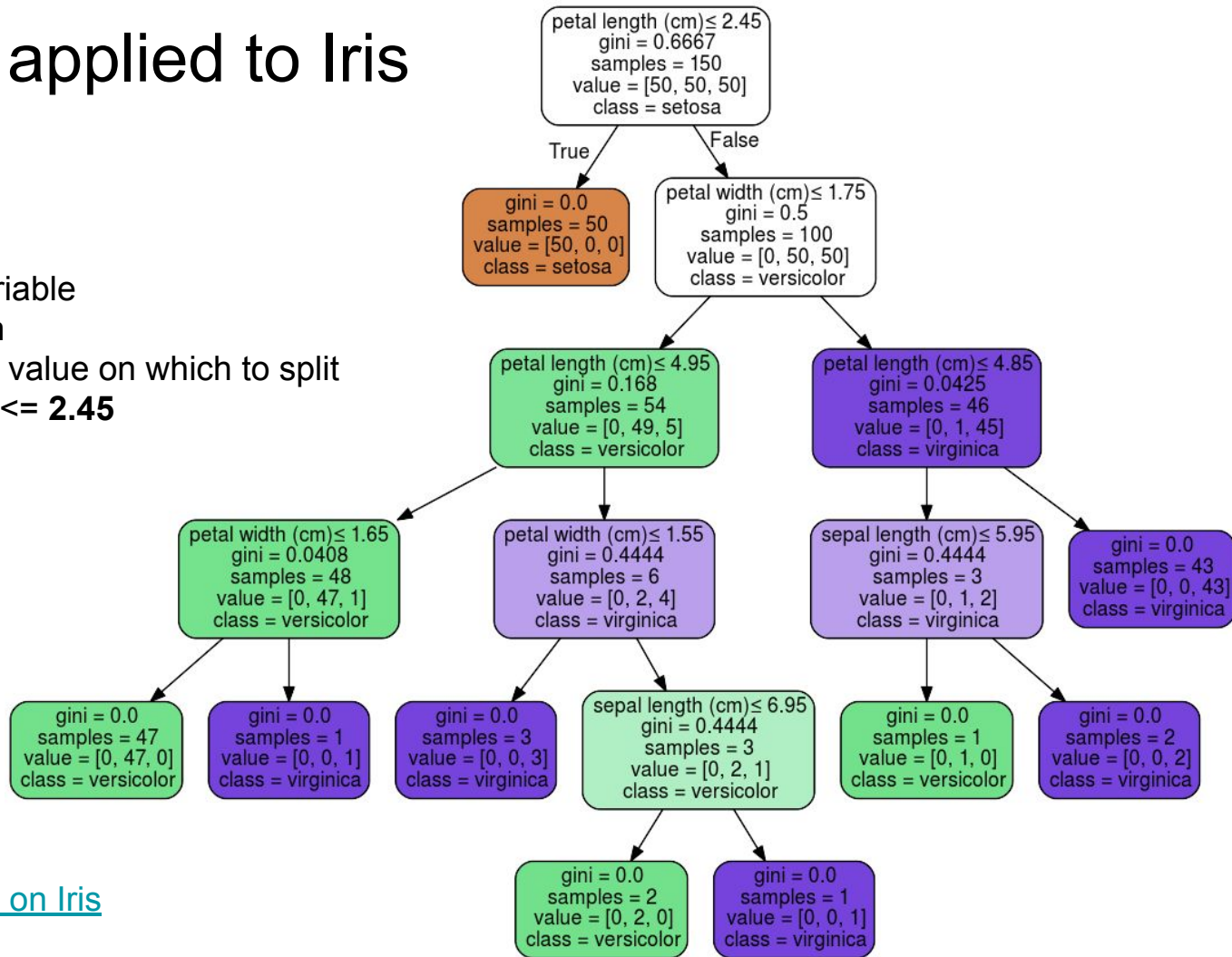
Decision trees - high variance estimators



Decision tree applied to Iris

On each iteration

1. Select the split variable
 - **petal length**
2. Find the threshold value on which to split
 - petal length ≤ 2.45



Notebook : [Decision tree on Iris](#)

Decision trees - sklearn

```
class sklearn.tree. DecisionTreeClassifier (criterion='gini', splitter='best', max_depth=None,  
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None, random_state=None,  
max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None, class_weight=None, presort=False)
```

[\[source\]](#)

- **split criterion** : gini, entropy, log loss
- **max_depth** : depth of the tree
- **min_samples_split** : min number of samples required to split an internal node
- **min_sample_leaf** : The min number of samples required to be at a leaf node.
- Main scikit page on Tree: <https://scikit-learn.org/stable/modules/tree.html>
- [Decision tree on Iris](#)

Splitting criterias

Gini Impurity

- **Formula:** $Gini = 1 - \sum (p_i^2)$
- **Range:** 0 (pure) to 0.5 (impure for binary)
- **Properties:**
 - Computationally efficient
 - Favors larger partitions
 - Default in CART algorithm

Entropy (Information Gain)

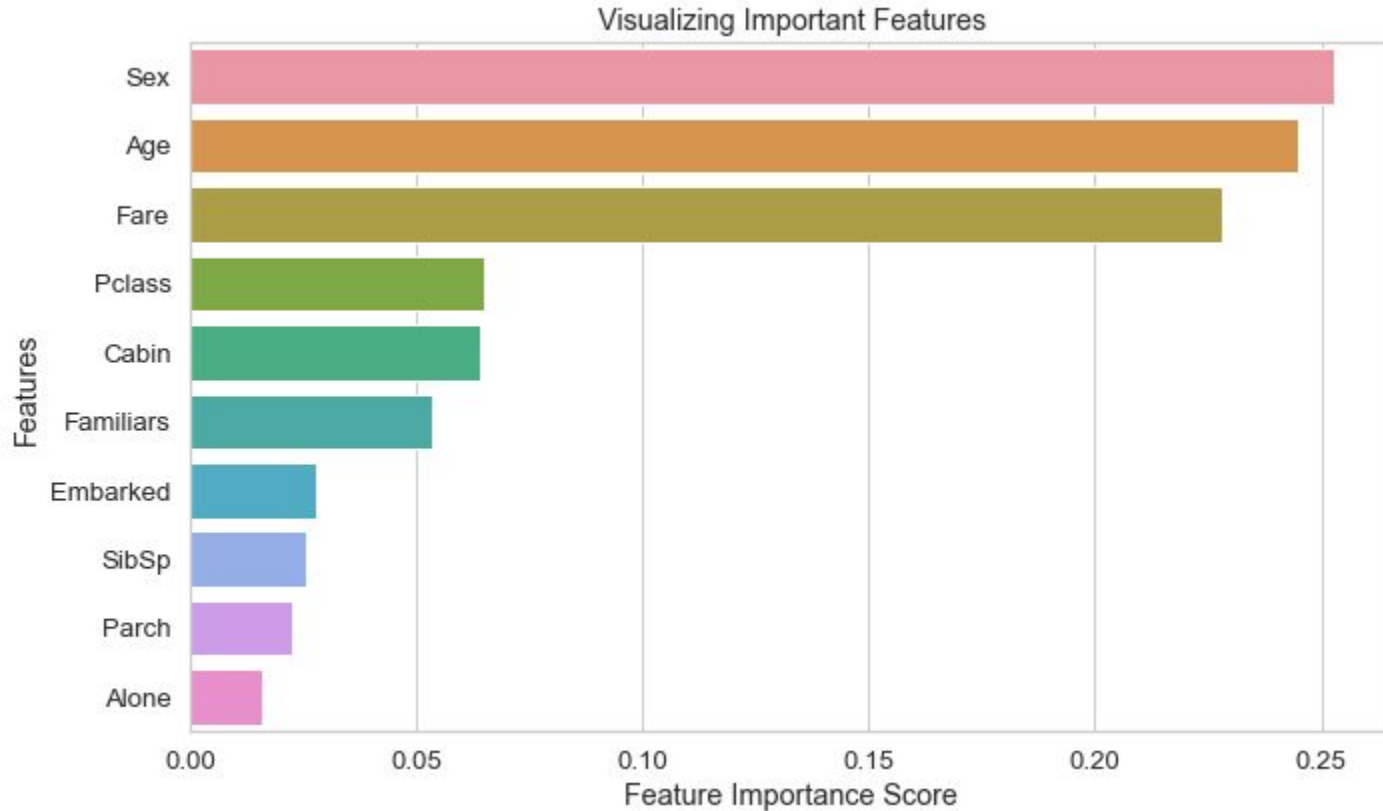
- **Formula:** $Entropy = -\sum (p_i \times \log_2(p_i))$
- **Range:** 0 (pure) to 1 (impure for binary)
- **Properties:**
 - Uses logarithms (more computation)
 - Creates more balanced trees
 - Default in ID3 and C4.5 algorithms

Log Loss (Cross-Entropy)

- **Formula:** $Log Loss = -\sum (y_i \times \log(p_i))$
- **Range:** 0 (perfect) to ∞ (terrible)
- **Properties:**
 - More common as evaluation metric
 - Sensitive to probability distributions
 - Heavily penalizes confident wrong predictions

Criterion	Computation	Tree Structure	Origin
Gini	Fastest	Often unbalanced	Economics
Entropy	Moderate	More balanced	Information theory
Log Loss	Slowest	Probability-sensitive	Statistical estimation

Decision trees - feature importance



`model.feature_importances_`

Decision trees - your turn

Train a decision tree on the penguins dataset

[data/penguins/penguins_size.csv](#)

- split train, test
- vary the depth from 2 to None, keep other parameters by default

For each model:

- observe the AUC and accuracy on the train and the test subset
 - observe the distribution of the proba for each category (histogram)
 - plot the feature importance
 - Visualize the tree structure
-
- What can you conclude in terms of overfitting ?

Decision trees - penguins dataset

Train a decision tree on the penguins dataset

<https://colab.research.google.com/drive/1jfNEDGZhS7DEQuTk3lgEd7zZt5bBcS2x?usp=sharing>

Ensemble methods

Weak learners - ensemble methods

A **weak learner** is a model with either (or both)

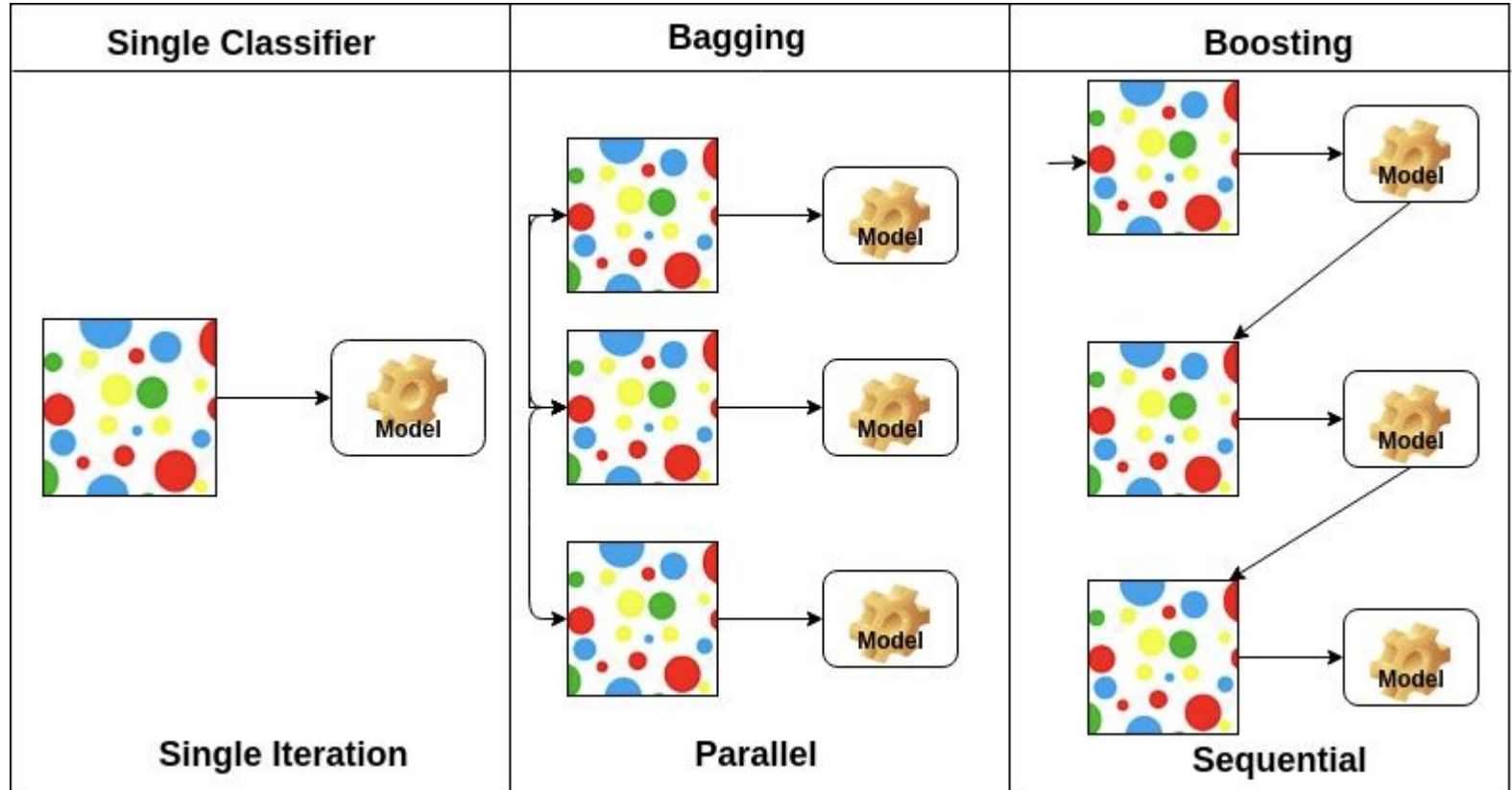
- high bias
- high variance (overfit)

The idea of **ensemble methods** is to combining several weak learners together in order to create a strong learner (or ensemble model) that achieves better performances.

Ensemble models are used to :

- decrease the variance for bagging (Bootstrap Aggregating) technique
- reduce bias for the boosting technique
- improving the predictive force for stacking technique.

ensemble methods: Bagging and Boosting



Stacking

Heterogeneous: mix plusieurs types de modèles

- Ridge regression + random forest
- Logistic regression + KNN
- ...

Bagging

Bagging: Bootstrap aggregating

The Goal: Reduce Variance

The idea is:

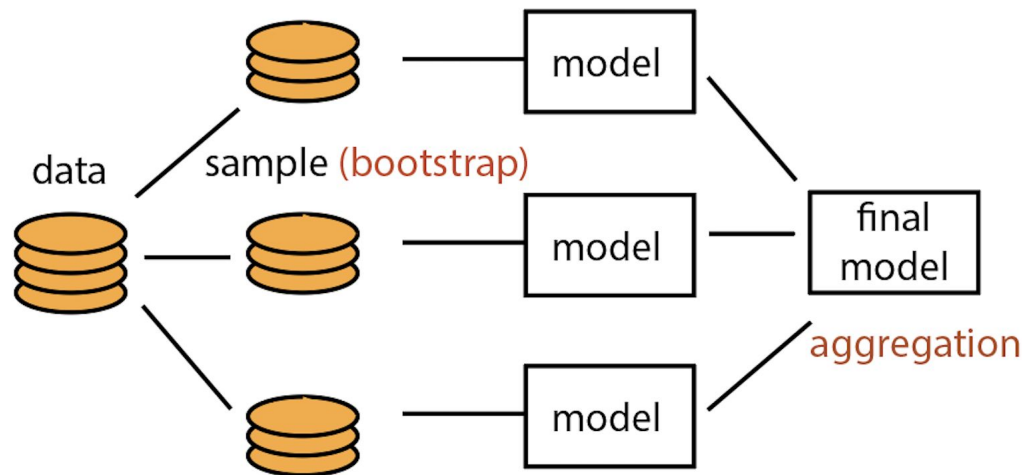
- Take a model with **high variance** and **low bias** and train it N times on a **bootstrap** of the data
 - This produces N trained models
- **Aggregate** the outputs from each of the N models to get the final result
 - **By voting**: most frequently predicted class (classification)
 - **By averaging** the predicted values (regression) or class probabilities (classification) =>
This technique reduces variance without degrading bias

Bagging : Bootstrap aggregating

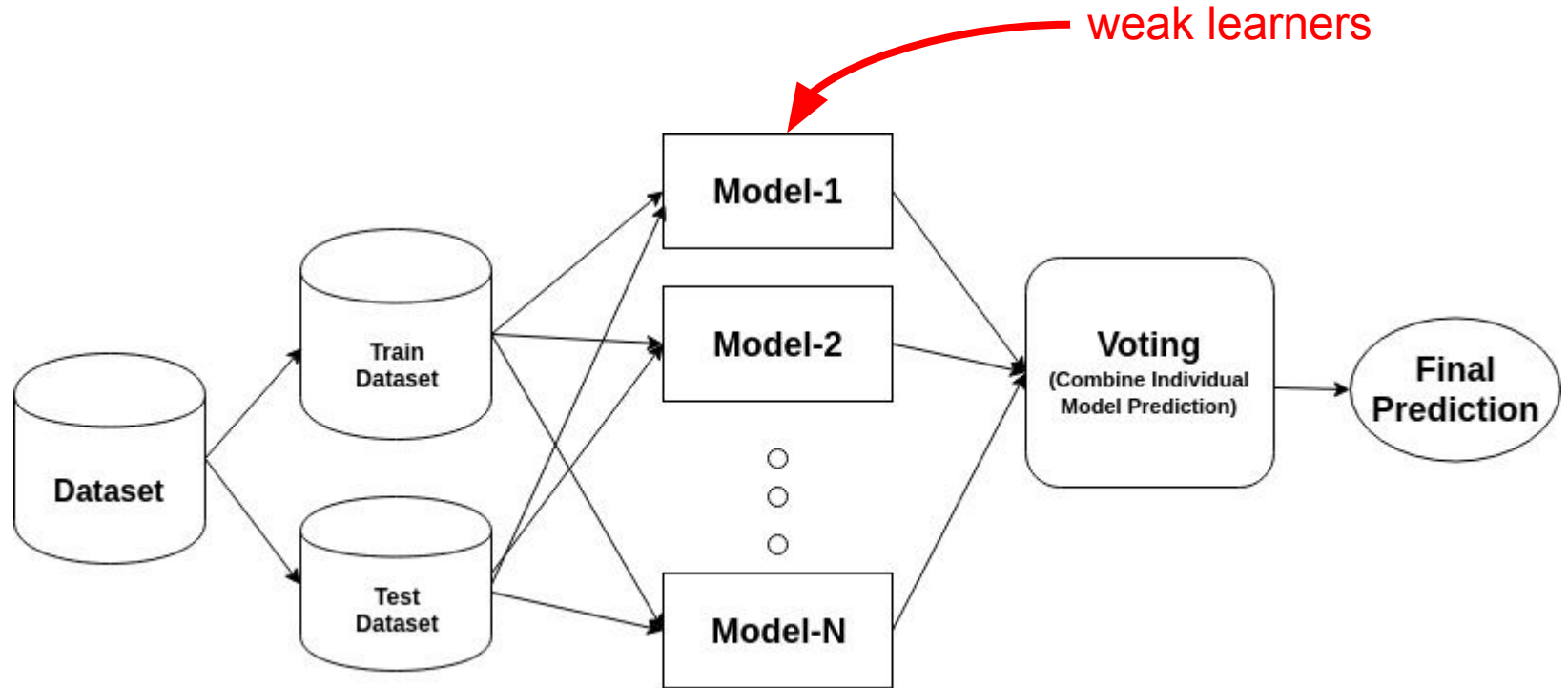
1. Bootstrap the training dataset (sample with replacement)
2. On each subset, train a weak learner (unconstrained overfitting tree)
3. Aggregate the predictions of all weak learners
 - regression: mean of predictions
 - classification
 - mean of probabilities (soft voting)
 - vote on predicted class

Excellent technique
to reduce
Overfitting

Bagging (Bootstrap aggregation)



Bagging: Bootstrap aggregating



Bagging: scikit

[sklearn.ensemble.BaggingClassifier — scikit-learn 0.23.2 documentation](#)

“A Bagging classifier is an ensemble meta-estimator that fits base classifiers each on random subsets of the original dataset and then aggregate their individual predictions (either by voting or by averaging) to form a final prediction.

Such a meta-estimator can typically be used as a way to reduce the variance of a black-box estimator (e.g., a decision tree), by introducing randomization into its construction procedure and then making an ensemble out of it.”

```
class sklearn.ensemble. BaggingClassifier(base_estimator=None, n_estimators=10, *, max_samples=1.0, max_features=1.0, bootstrap=True, bootstrap_features=False, oob_score=False, warm_start=False, n_jobs=None, random_state=None, verbose=0)
```

[\[source\]](#)

Your turn – Bagging

data : tbd

Create the train and test subsets

- use a simple decision tree, unpruned (max depth = None) as a base learner.
- train the single decision tree, and check the accuracy on train and test sets. This is your benchmark
- Take 20 trees, unpruned
- for each tree, predict the probabilities on the test set
- average the probabilities and use the average to determine the predicted class
- what is the accuracy on the test set ?

Now using sklearn.tree. **BaggingClassifier** with the estimator as the single unpruned tree

Now use a single tree with max depth of 2 as the base model (high bias) as your benchmark

- verify that the model has high bias
- use the bagging classifier with 20 trees
- is the bias reduced ? what about the variance ?

Bagging: Démo

Sur le dataset Titanic

- Split train validation 70 / 30

Répéter N fois

- Bootstrap les données D^{train_k}
- Train: decisionTreeClassifier $T_k(D^{\text{train}_k})$ sans pruning

Puis

- Appliquer les N arbres sur le dataset de validation
 - On a pour chaque échantillon, N résultats.
- Pour chaque échantillon, prendre la classe la plus fréquente parmi les N résultats
- Calculer le score du modèle globale: matrice de confusion, AUC, ...

Ensuite:

- Prendre un autre classifieur faible (LR)
- Estimer sa performance quand utilisé seul.
- Utiliser Bagging classifieur de scikit avec différents classifieurs simples.

Voir http://localhost:8888/notebooks/2018_19_Emines/notebooks/05%20bagging.ipynb

Bagging: scikit

Extra Practice: <https://machinelearningmastery.com/bagging-ensemble-with-python/>

- Explore number of learners
- Explore sample size
- Explore other classifiers
- Explore high variance low bias vs low variance high bias

See also bagging with regression trees on Ames Housing dataset:

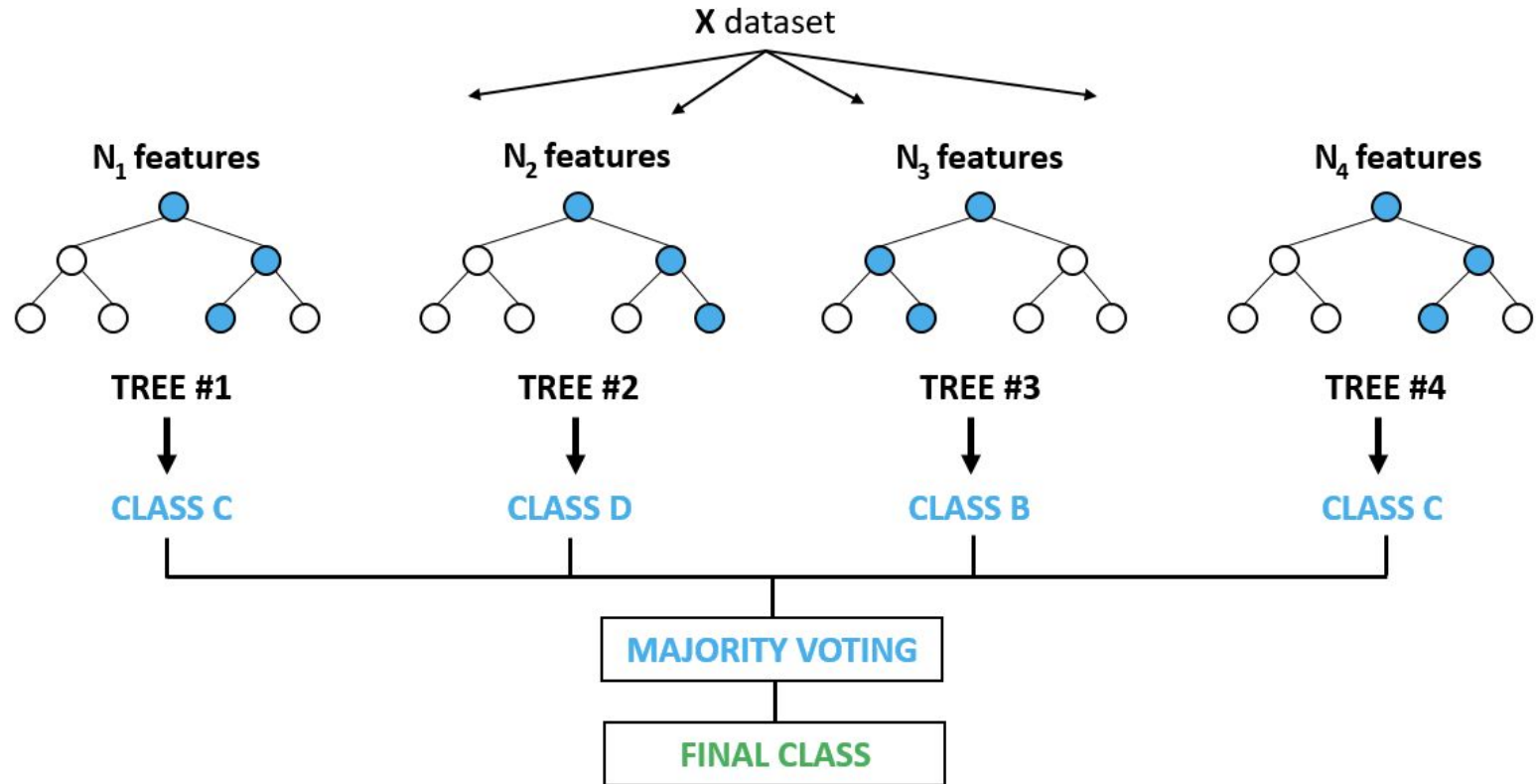
<https://colab.research.google.com/drive/1M8H8G3QLd7k1rVuZ5yYiSyN1U1lbzswR?usp=sharing>

Random Forests

Random forests

1. Decision trees
 2. **Bagging** : average the models trained on bootstrapped data
 3. **Random Forest** => Bagging(Trees) + random features selection
- A **Random Forest** is a **generalization of Bagging that is specific to Decision Trees**.
 - At each branch in the decision tree, Random Forest subsamples
 - the training data.
 - the features
 - Intuitively, this process further de-correlates the individual trees, which is good for Bagging.

Random Forests



Random forests

[3.2.4.3.1. sklearn.ensemble.RandomForestClassifier — scikit-learn 0.23.2 documentation](#)

```
class sklearn.ensemble.RandomForestClassifier(n_estimators=100, *, criterion='gini', max_depth=None,  
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='auto', max_leaf_nodes=None,  
min_impurity_decrease=0.0, min_impurity_split=None, bootstrap=True, oob_score=False, n_jobs=None, random_state=None,  
verbose=0, warm_start=False, class_weight=None, ccp_alpha=0.0, max_samples=None)
```

[\[source\]](#)

[3.2.4.3.2. sklearn.ensemble.RandomForestRegressor — scikit-learn 0.23.2 documentation](#)

```
class sklearn.ensemble.RandomForestRegressor(n_estimators=100, *, criterion='mse', max_depth=None,  
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='auto', max_leaf_nodes=None,  
min_impurity_decrease=0.0, min_impurity_split=None, bootstrap=True, oob_score=False, n_jobs=None, random_state=None,  
verbose=0, warm_start=False, ccp_alpha=0.0, max_samples=None)
```

[\[source\]](#)

Random forests: OOB

OUT OF BAG - OOB

When bootstrapping, in each experiment will use only approx. $2/3$ rd of the available samples.

Which leaves $1/3$ rd that we can use to estimate the validation error of each tree.

This is called OOB: Out of Bag error.

It can be shown that with B sufficiently large, OOB error is virtually equivalent to **leave-one-out** cross-validation error.

Random forests: OOB

With B sufficiently large, OOB error is virtually equivalent to **leave-one-out** cross-validation error.

Understanding Out-of-Bag (OOB) Error

- **Bootstrap sampling:** Each tree in an ensemble uses random sampling with replacement
 - ~63% of data used for training each tree ("in the bag")
 - ~37% of data not used for each tree ("out of the bag")
- **OOB prediction:** For each observation, aggregate predictions only from trees that never saw it during training
- **OOB error:** The error rate calculated on these "unseen" predictions

The Mathematics Behind the Equivalence

- **Probability theory:** With B bootstrap samples, probability that an observation is never selected in a particular bootstrap sample: $(1 - 1/n)^n \approx e^{-1} \approx 0.368$
- **Convergence property:** As B increases, each observation is OOB in approximately 37% of trees
- **Statistical consistency:** The OOB prediction distribution approaches the leave-one-out prediction distribution
- **Error estimation:** Both methods evaluate model performance on data points not used in training

Three Ways to Measure Feature Importance

1. Mean Decrease in Impurity (MDI)

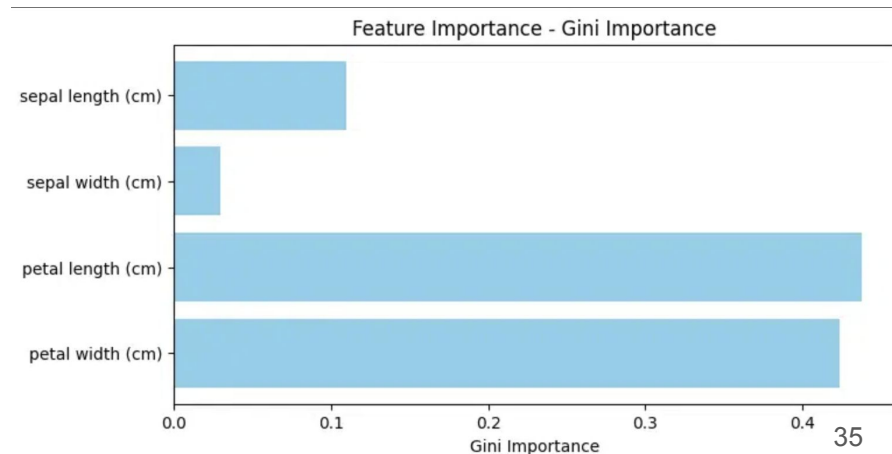
- Measures how much each feature reduces node impurity across all trees
- Built-in, computed during training
- **Limitation:** Biased toward high cardinality features

2. Permutation Importance

- Randomly shuffle one feature's values and measure performance drop
- More reliable measure of actual prediction impact
- **Limitation:** Computationally expensive

3. Drop-Column Importance

- Remove feature entirely and retrain model
- Most realistic measure but requires complete retraining
- **Limitation:** Very computationally expensive

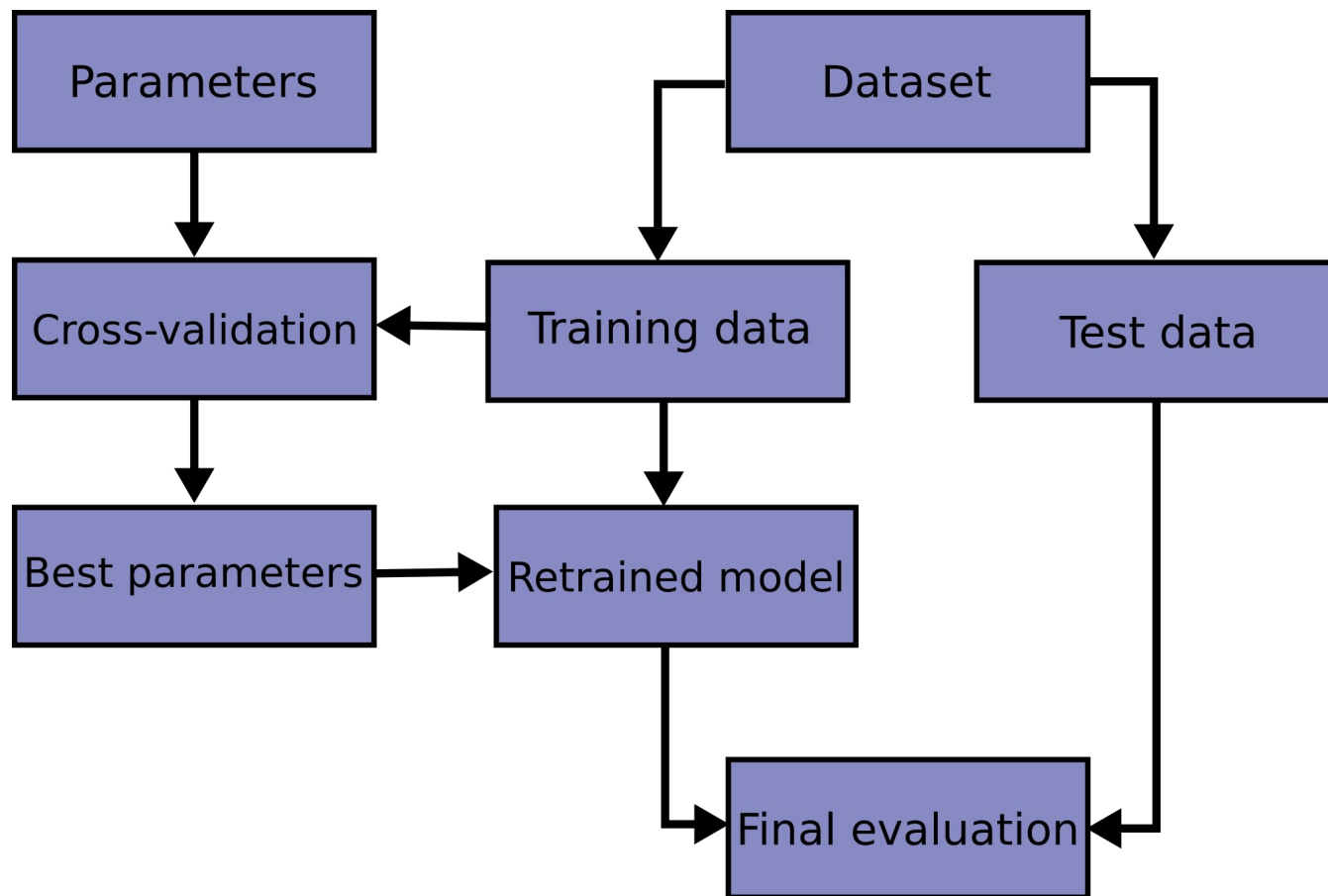


Train a simple Random forest - regression

- dataset, train, test split
- tune the `max_depth` and `n_features` with `gridsearchCV`
- observe the RMSE and R^2
 - check for overfitting
- what's the lowest score you can get ?

Model selection

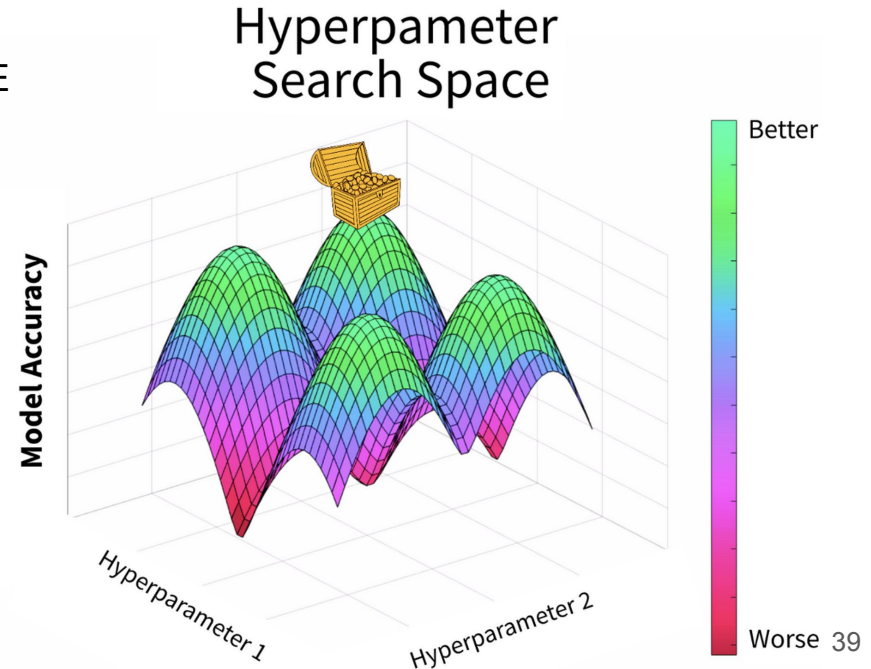
Model selection and parameter optimization



Hyperopt - bayesian parameter selection

What is Hyperopt?

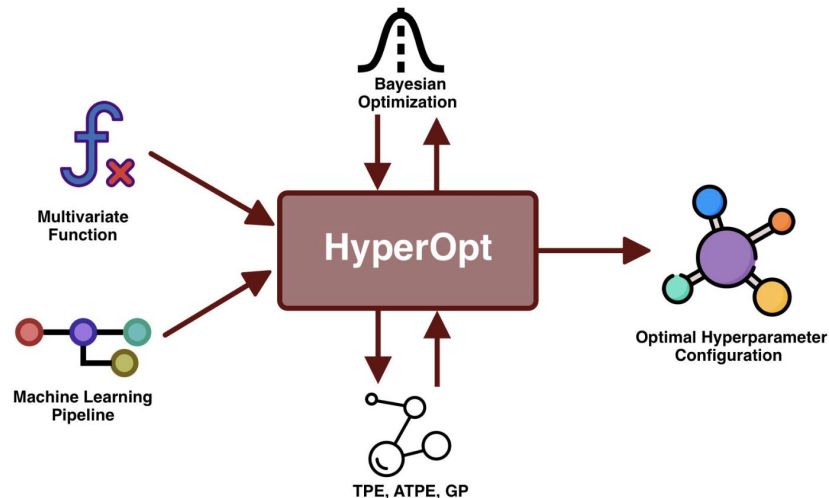
- Python library for optimizing hyperparameters of machine learning algorithms
- Uses Bayesian optimization techniques (primarily TPE - Tree of Parzen Estimators)
- Efficiently searches complex parameter spaces
- **More efficient than grid search or random search**
- Learns from previous evaluations to focus on promising regions



Hyperopt - bayesian parameter selection

Core Components

- **Search Space Definition:** `hp` module with sampling distributions
- **Objective Function:** Function to minimize (e.g., validation error)
- **Optimization Algorithm:** TPE, Random Search, etc.
- **Trials Object:** Stores history of evaluations
- **fmin():** The main optimization function



Hyperopt - bayesian parameter selection

Method	Description	Example	Common Use Case
<code>hp.choice(label, options)</code>	Discrete selection from a list	<code>hp.choice('activation', ['relu', 'tanh', 'sigmoid'])</code>	Categorical parameters
<code>hp.uniform(label, low, high)</code>	Continuous uniform distribution	<code>hp.uniform('learning_rate', 0.0001, 0.1)</code>	Learning rates, regularization
<code>hp.quniform(label, low, high, q)</code>	Quantized uniform distribution	<code>hp.quniform('max_depth', 1, 30, 1)</code>	Integer parameters with quantization
<code>hp.loguniform(label, low, high)</code>	Log-uniform distribution	<code>hp.loguniform('learning_rate', np.log(1e-5), np.log(1))</code>	Learning rates (log scale)
<code>hp.normal(label, mu, sigma)</code>	Normal distribution	<code>hp.normal('weight_decay', 0, 1)</code>	Parameters centered around a value

Hyperopt - define the search space

```
from hyperopt import hp

space = {
    # Integer parameters
    'n_estimators': hp.quniform('n_estimators', 10, 500, 10),
    'max_depth': scope.int(hp.quniform('max_depth', 1, 20, 1)),

    # Continuous parameters
    'learning_rate': hp.loguniform('learning_rate', np.log(0.001), np.log(0.1)),

    # Categorical parameters
    'criterion': hp.choice('criterion', ['gini', 'entropy', 'log_loss']),
    'max_features': hp.choice('max_features', ['auto', 'sqrt', 'log2']),

    # Boolean parameters
    'bootstrap': hp.choice('bootstrap', [True, False])
}
```

Hyperopt - define an objective function

```
def objective(params):
    # Create model with given hyperparameters
    model = RandomForestClassifier(
        n_estimators=int(params['n_estimators']),
        max_depth=int(params['max_depth']),
        # Other parameters...
        random_state=42
    )

    # Evaluate model using cross-validation
    cv_scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')
    mean_cv_score = cv_scores.mean()

    # Return dict with 'loss' to be minimized
    return {
        'loss': -mean_cv_score,  # Negative because we want to maximize accuracy
        'status': STATUS_OK,
        'model': model
    }
```

Hyperopt - Compare with Grid search

Benefits Over Traditional Methods

Method	Efficiency	Handles Complex Spaces	Learns from History
Grid Search	✗ Poor	✗ Limited	✗ No
Random Search	⚠ Medium	✓ Yes	✗ No
Hyperopt	✓ Excellent	✓ Yes	✓ Yes

- Finds better hyperparameters in fewer iterations
- Scales well to high-dimensional parameter spaces
- Particularly effective for expensive evaluation functions

Hyperopt - Demo

https://colab.research.google.com/drive/1uNXcpFk3-ZTggaINaOX_BbGS2sbcvXRh

Optuna

<https://optuna.org/>

Random forests - Practice

- Pick a classification dataset (can be artificial with `make_classification`)
 - split in 2 subsets : main, valid (only apply the model to valid after parameter selection)
- Find optimal parameters :
 - `n_estimators`, `max_depth`, `max_features`, `bootstrap`
 - influence of splitting criteria
- Difference between `GridSearchCV` and `RandomizedSearchCV`
- Cross-Validation on main subset
- Explore the best model: feature importance, best parameters
- Once the model is optimized
 - retrain the model on the whole main dataset
 - Observe on valid
 - confusion matrix
 - histogram of probabilities of predictions on valid
 - what samples are not properly predicted : False Positives and False Negatives
 -

Lectures et ressources

Bagging

- Ensembling methods on scikit
[1.11. Ensemble methods — scikit-learn 0.23.2 documentation](#)
- [How to Develop a Bagging Ensemble with Python](#)

Trees

- [Coding a Random Forest from Scratch in Python p.2: Bootstrapping and Random Subspace Method](#) and [Coding a Random Forest from Scratch](#)
- [How to Implement Random Forest From Scratch in Python](#)
- [Random forests and decision trees from scratch in python](#)
- [Building a Random Forest from Scratch In Python](#)
- [Implementing a classification tree with Gini Impurity from scratch in Python | by Pandula Weerasooriya](#)
- good explanation of trees and GINI and illustration with scikit learn
[Understanding Decision Trees for Classification in Python](#)
- gini, scikit with visualization graphviz
[Decision Tree Classification in Python](#)